4. Sparse Singular Value Decomposition

Lieven Clement

Invalid Date

Table of contents

1 Introduction **1**
1.1 Example: Toxicogenomics in early drug development . . . 2
1.1.1 Background . . . 2
1.1.2 Data . . . 2
1.1.3 Data exploration . . . 3
1.2 Sparse matrix decomposition . . . 6

2 Penalised Matrix Decomposition **6**
2.1 Rank 1 approximation . . . 6
2.2 Rank-2 approximation . . . 8
2.3 Rank-k approximation . . . 8
2.4 Sparse PCA via PMD . . . 9

3 Toxicogenomics analysis with a sparse PCA **9**
3.1 SPCA . . . 9
3.2 Loading . . . 11
3.3 Change constraint . . . 12
3.4 Variance explained . . . 16

Acknowledgement **20**

Session info **20**

1 Introduction

In high dimensional data the PCs may have unclear interpretation because they depend on all p original variables.

1.1 Example: Toxicogenomics in early drug development

1.1.1 Background

- Effect of compound on gene expression.
- Insight in action and toxicity of drug in early phase
- Determine activity with bio-assay: e.g. binding affinity of compound to cellwall receptor (target, IC50).
- Early phase: 20 to 50 compounds
- Based on in vitro results one aims to get insight in how to build better compound (higher on-target activity less toxicity.
- Small variations in molecular structure lead to variations in BA and gene expression.
- Aim: Build model to predict bio-activity based on gene expression in liver cell line.

1.1.2 Data

- 30 chemical compounds have been screened for toxicity
- Bioassay data on toxicity screening
- Gene expressions in a liver cell line are profiled for each compound (4000 genes)

```
toxData <- read_csv(
  "https://raw.githubusercontent.com/statOmics/HDA2020/data/toxDataCentered.csv",
  col_types = cols()
)
svdX <- svd(toxData[,-1])
```

Data is already centered:

```
toxData |>
  colMeans() |>
  range()
```

```
#> [1] -2.405483e-17  5.921189e-17
```

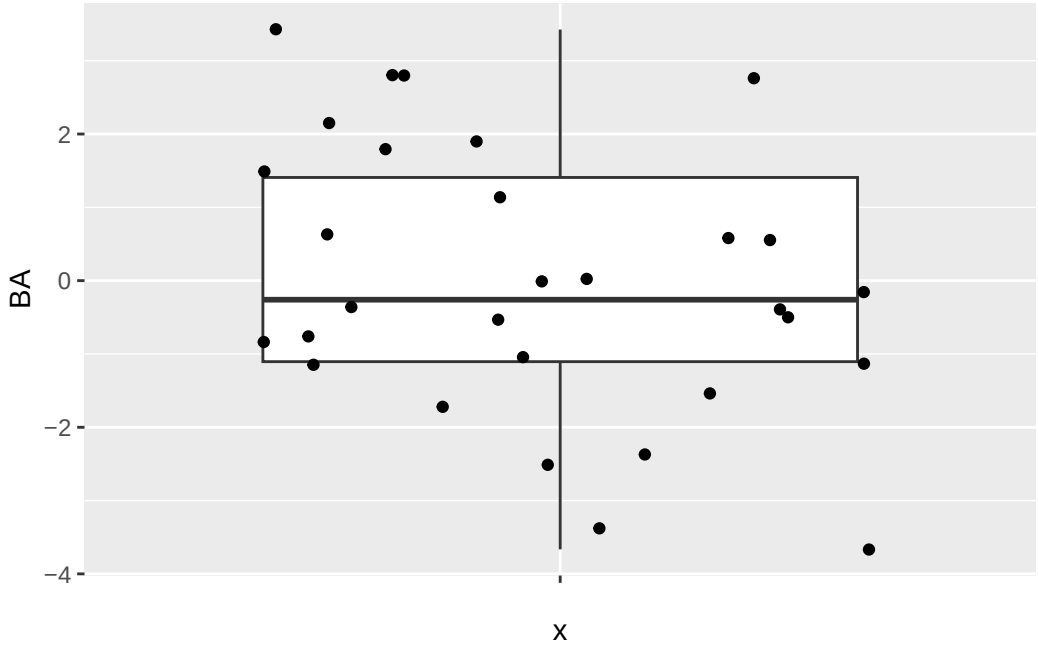
```
 toxData |>
  names() |>
  head()
```

```
#> [1] "BA" "X1" "X2" "X3" "X4" "X5"
```

- First column contains data on Bioassay.
- The higher the score on Bioassay the more toxic the compound
- Other columns contain data on gene expression X1, … , X4000

1.1.3 Data exploration

```
toxData |>
  ggplot(aes(x="",y=BA)) +
  geom_boxplot(outlier.shape=NA) +
  geom_point(position="jitter")
```

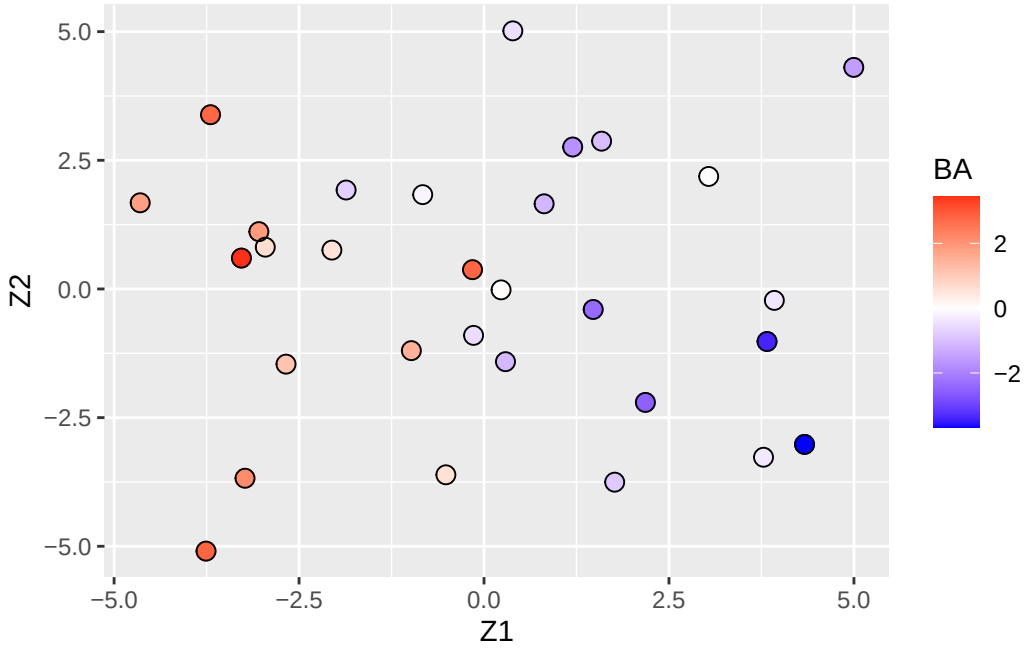



```
svdX <- toxData[,-1] |>
  svd()

k <- 2
Vk <- svdX$v[,1:k]
Uk <- svdX$u[,1:k]
Dk <- diag(svdX$d[1:k])
Zk <- Uk%*%Dk
```

```r
colnames(Zk) <- paste0("Z",1:k)
colnames(Vk) <- paste0("V",1:k)

pca <- Zk |>
  as.data.frame() |>
  mutate(BA = toxData |> pull(BA)) |>
  ggplot(aes(x= Z1, y = Z2, color = BA)) +
  geom_point(size = 3) +
  scale_colour_gradient2(low = "blue",mid="white",high="red") +
  geom_point(size = 3, pch = 21, color = "black")
pca
```

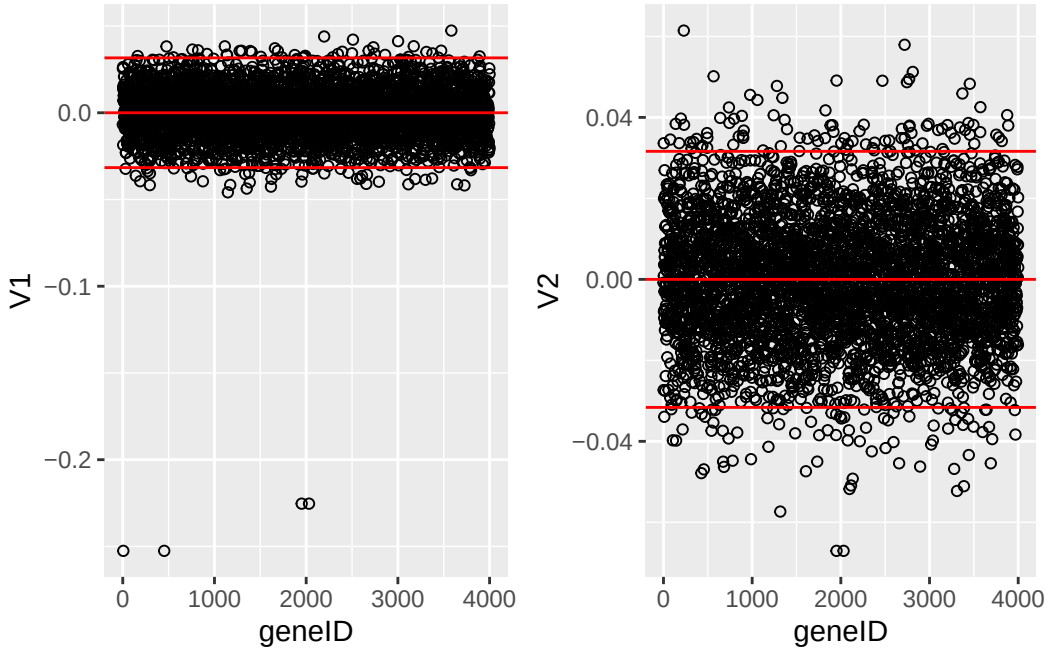


- Scores on the first two principal components (or MDS plot). - Each point corresponds to a compound.
- Color code refers to the toxicity score (higher score more toxic).
- Clear separation between compounds according to toxicity.

- Next logic step in a PCA is to interpret the principal components.
- We thus have to assess the loadings.
- We can add a vector for each gene to get a biplot, but this would require plotting 4000 vectors, which would render the plot unreadable.

Alternative graph to look at the many loadings of the first two PCs.

```
grid.arrange(
  Vk |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vk)) |>
    ggplot(aes(x = geneID, y = V1)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vk[,1]), col = "red") ,
  Vk |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vk)) |>
    ggplot(aes(x = geneID, y = V2)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vk[,2]), col = "red"),
  ncol=2)
```



- It is almost impossible to interpret the PCs because there are 4000 genes contributing to each PC.
- In an attempt to find the most important genes (in the sense that they drive the interpretation of the PCs), the plots show horizontal reference lines: the average of the loadings, and the average $\pm$ twice the standard deviation of the loadings. In between the lines we expects about 95% of the loadings (if they were normally distributed).

- The points outside the band come from the genes that have rather large loadings (in absolute value) and hence are important for the interpretation of the PCs.
- Note, that particularly for the first PC, only a few genes show a markedly large loadings that are negative. This means that an upregulation of these genes will lead to low scores on PC1.
- These genes will very likely play an important role in the toxicity mechanism.
- Indeed, low scores on PC1 are in the direction of more toxicity.

1.2 Sparse matrix decomposition

Basic idea of sparse matrix decomposition of an $n \times p$ matrix $\mathbf{X}\}$

Find a rank k approximation of $\mathbf{X}$ of the form

$$\mathbf{X}_k = \sum_{j=1}^{k} \delta_j \mathbf{u}_j \mathbf{v}_j^t$$

such that many of the elements in the $\mathbf{v}_j$ and or $\mathbf{u}_j$ are exactly zero.

We focus on the **Penalised Matrix Decomposition** (PMD) method of Witten *et al.* (2009, Biostatistics, 10, 3, 515-534).

When many of the loadings in $\mathbf{v}_j$ are zero, the interpretation of the PC only depends on the features that correspond to the remaining non-zero loadings.

---

2 Penalised Matrix Decomposition

2.1 Rank 1 approximation

The rank-1 PMD approximation of the $n \times p$ matrix $\mathbf{X}$ is of the form

$$\mathbf{X}_1 = \delta_1 \mathbf{u}_1 \mathbf{v}_1^t,$$

and it is the solution to the following minimisation problem:

$$\min_{\delta,u,v} \|\mathbf{X} - \delta \mathbf{u}\mathbf{v}^t\|_F^2 \text{ subject to } \delta \geq 0, \|\mathbf{u}\|_2^2 = \|\mathbf{v}\|_2^2 = 1, \|\mathbf{u}\|_1 \leq c_1, \|\mathbf{v}\|_1 \leq c_2$$

for some user-defined constants $c_1, c_2 > 0$.

The constraints $\|\mathbf{u}\|_1 \leq c_1$ and $\|\mathbf{v}\|_1 \leq c_2$ are known as L_1 or lasso constraints.

Note,

$$\|\mathbf{u}\|_1 = \sum_{i=1}^{n} |u_i| \text{ and } \|\mathbf{v}\|_1 = \sum_{i=1}^{p} |v_i|.$$

If $c_1 = +\infty$, then the constraint on $\mathbf{u}$ is removed (similar for c_2).

Note that

$$\min_{\delta,u,v} \|\mathbf{X} - \delta \mathbf{u}\mathbf{v}^t\|_F^2 = \min_{A:\text{rank}(A)=1} \|\mathbf{X} - A\|_F^2$$

with $A = X_1$ (truncated SVD of $\mathbf{X}$, truncated after 1 term). However, the PMD requires a constrained minimisation. The unconstrained solution arises with $c_1 = c_2 = +\infty$.

Note that $\|\mathbf{u}\|_2^2 = \|\mathbf{v}\|_2^2 = 1$ may also be written as $\mathbf{u}^t\mathbf{u} = \mathbf{v}^t\mathbf{v} = 1$ (i.e. the vectors are normalised, just like for the singular vectors of the SVD).

- We also applied the L_1 constraint to the least-squares estimator of the regression parameters in a linear regression model.
- Effect was that many of the parameter estimates are set exactly to zero.
- Here, the same effect will take place. Many of the elements in $\mathbf{u}$ and $\mathbf{v}$ will be set exactly to zero.
- In particular, the smaller c_2, the more elements of $\mathbf{v}$ will be set to zero (shrinkage).
- Similarly, the smaller c_1, the more elements of $\mathbf{u}$ will be set to zero.
- The choice of c_1 and c_2 depends on the objective of the data analysis and will be discussed later.

The solution is known as a rank-1 PMD approximation of $\mathbf{X}$ because $\delta_1 \mathbf{u}_1 \mathbf{v}_1^t$ is a matrix of rank 1.

2.2 Rank-2 approximation

The rank-2 PMD approximation is found as follows:

1. find the rank-1 PMD as before: $\mathbf{X}_1 = \delta_1 \mathbf{u}_1 \mathbf{v}_1^t$;
2. compute
$$\tilde{\mathbf{X}}_2 = \mathbf{X} - \mathbf{X}_1,$$
and compute the rank-1 PMD of $\tilde{\mathbf{X}}_2$, which gives $\delta_2, \mathbf{u}_2, \mathbf{v}_2$;
3. the rank-2 PMD of $\mathbf{X}$ is then given by
$$\mathbf{X}_2 = \delta_1 \mathbf{u}_1 \mathbf{v}_1^t + \delta_2 \mathbf{u}_2 \mathbf{v}_2^t.$$

Note that the $\mathbf{u}$'s and $\mathbf{v}$'s are not necessarily orthogonal (only if $c_1, c_2 = +\infty$).

2.3 Rank-k approximation

The rank-k PMD approximation is found as follows:

1. find the rank-1 PMD as before: $\tilde{\mathbf{X}}_1 = \delta_1 \mathbf{u}_1 \mathbf{v}_1^t$;
2. set $j = 1$;
3. compute
$$\tilde{\mathbf{X}}_{j+1} = \mathbf{X} - \tilde{\mathbf{X}}_1 - \cdots - \tilde{\mathbf{X}}_j,$$
and compute the rank-1 PMD of $\tilde{\mathbf{X}}_{j+1}$, which gives $\delta_{j+1}, \mathbf{u}_{j+1}, \mathbf{v}_{j+1}$;
4. if $j < k$, increase j with one and go back to step 3; otherwise go to step 5;
5. the rank-k PMD of $\mathbf{X}$ is then given by
$$\mathbf{X}_k = \sum_{j=1}^{k} \delta_j \mathbf{u}_j \mathbf{v}_j^t.$$

Without the L_1 constraints, this algorithm gives the ordinary SVD.

As before the δ_js, $\mathbf{u}_j$s and $\mathbf{v}_j$s are stacked into matrices $\mathbf{D}_k$, $\mathbf{U}_k$ and $\mathbf{V}_k$: $\mathbf{X}_k = \mathbf{U}_k \mathbf{D}_k \mathbf{V}_k^t$.

Note that the $\mathbf{u}$ vectors are generally not orthogonal (i.e. $\mathbf{u}_i^t \mathbf{u}_j \neq 0$ for $i \neq j$) (the same holds for the $\mathbf{v}$ vectors).

2.4 Sparse PCA via PMD

Just like the SVD can be used to find the PCA solution, the PMD can be used to find a sparse PCA solution (SPC).

For a PCA, it is only important to have sparse loadings (in the **v**s); the scores (involving the **u**s) need not be sparse.

The rank-1 solutions thus satisfy

$$\min_{\delta,u,v} \|\mathbf{X} - \delta\mathbf{u}\mathbf{v}^t\|_F^2 \text{ subject to } \delta \geq 0, \|\mathbf{u}\|_2^2 = \|\mathbf{v}\|_2^2 = 1, \|\mathbf{v}\|_1 \leq c$$

(i.e. no sparsity constraint on **u**).

Hence the PCA scores

$$\mathbf{Z}_k = \mathbf{U}_k\mathbf{D}_k$$

relate to $\mathbf{X}_k$ through the sparse loadings in $\mathbf{V}_k$.

It can be shown that the SPC solution (rank-1) also maximises

$$\mathbf{v}^t\mathbf{X}^t\mathbf{X}\mathbf{v} \text{ subject to } \|\mathbf{u}\|_2^2 = \|\mathbf{v}\|_2^2 = 1, \|\mathbf{v}\|_1 \leq c$$

(i.e. it maximises the variance of the PCs, subject to sparsity constraint on loadings).

From these two slides we may conclude that we can use the PMD as the basis of a sparse PCA, and that the interpretation is as before in terms of maximising variance. Only now we hope that only a few features will give a non-zero loading in the **v**-vectors. Biplots may again be constructed (with fewer vectors to be plotted).

---

3 Toxicogenomics analysis with a sparse PCA

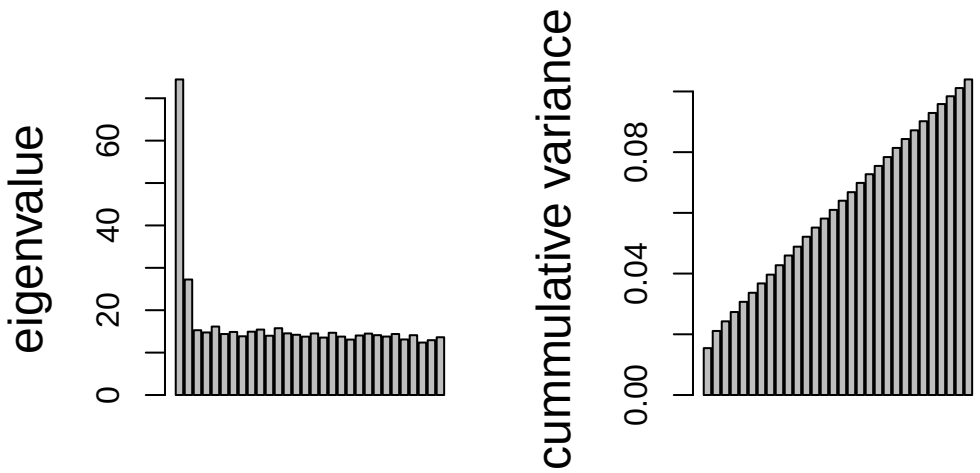
3.1 SPCA

```
library(PMA)

spcX <- SPC(
  toxData[,-1] |>
    as.matrix(),
  K=30,
  sumabsv = 5)
```

```
#> 12345678910111213141516171819
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
#> 1234567891011121314151617181920
```

```
par(mfrow=c(1,2))
barplot(spcX$d^2,ylab="eigenvalue",cex.lab=1.5)
barplot(spcX$prop.var.explained,
     ylab="cummulative variance",cex.lab=1.5)
```



- The plot on the left shows the eigenvalues (squared singular values) from the PMD.
  - This suggests that the first one or two are the most important, but all eigenvalues remain substantially larger than zero.
- The figure on the right shows the cumulative percentage of variance retained in the PCs.
  - These percentages are now no longer calculated as before as ratio's of sums of eigenvalues.
  - A more complicated calculation is required now, because the PCs are no longer uncorrelated with one another.
  - Thus part of the information (variance) of one PC is shared with part of the information of another PCs (i.e. non-zero correlation or covariance).

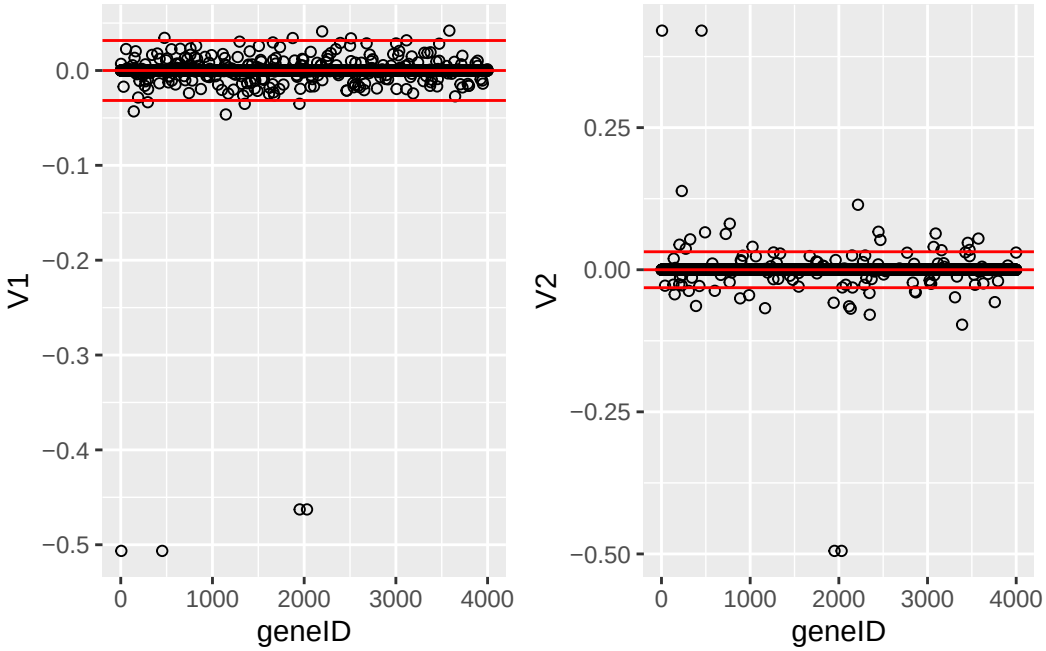
3.2 Loading

Loadings of the first two PCs, with constraint $\|\mathbf{v}\|_1 < 5$.

```
k <- 2
Vk <- spcX$v[,1:k]
colnames(Vk) <- paste0("V",1:k)

grid.arrange(
```

```
Vk |>
  as.data.frame() |>
  mutate(geneID = 1:nrow(Vk)) |>
  ggplot(aes(x = geneID, y = V1)) +
  geom_point(pch=21) +
  geom_hline(yintercept = c(-2,0,2)*sd(Vk[,1]), col = "red") ,
Vk |>
  as.data.frame() |>
  mutate(geneID = 1:nrow(Vk)) |>
  ggplot(aes(x = geneID, y = V2)) +
  geom_point(pch=21) +
  geom_hline(yintercept = c(-2,0,2)*sd(Vk[,2]), col = "red"),
ncol=2)
```



These plots show the loadings of the first two sparse PCs. Many of the loadings are now exactly equal to zero. Only few genes show large loadings.

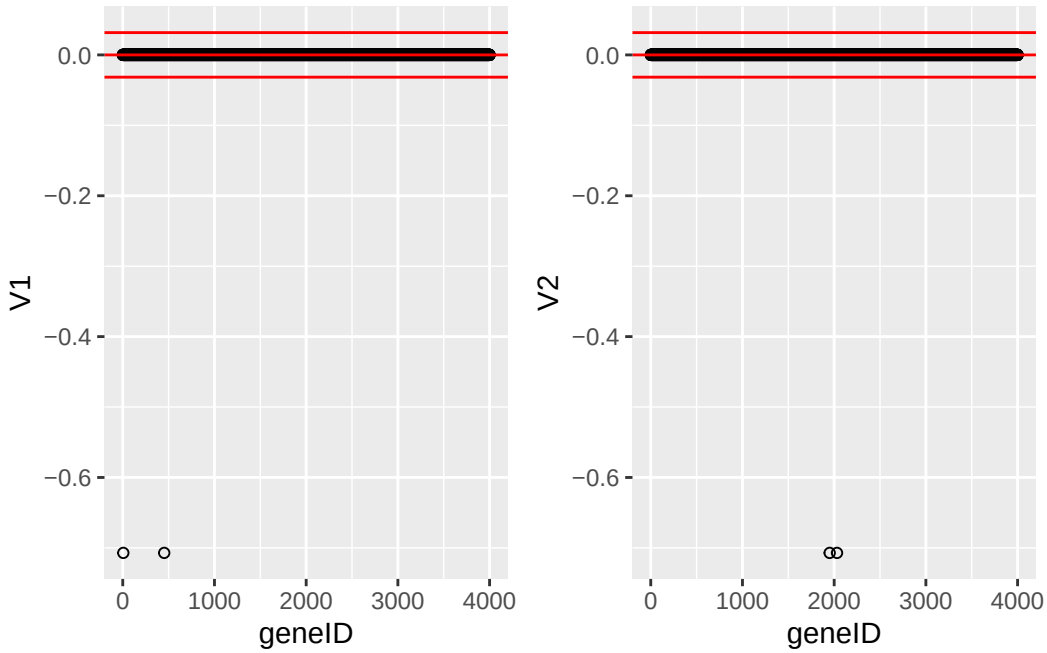
3.3 Change constraint

We now repeat the analysis, but with the constraint $\|\mathbf{v}\|_1 < 1$. This gives a much sparser solution with only two genes with non-zero loadings in each dimension.

```r
spcX1 <- SPC(
  toxData[,-1] |>
    as.matrix(),
  K=30,
  sumabsv = 1)
```

```
#> 12
#> 12
#> 12
```

```
Vk |>
  as.data.frame() |>
  mutate(geneID = 1:nrow(Vk)) |>
  ggplot(aes(x = geneID, y = V1)) +
  geom_point(pch=21) +
  geom_hline(yintercept = c(-2,0,2)*sd(Vk[,1]), col = "red") ,
Vk |>
  as.data.frame() |>
  mutate(geneID = 1:nrow(Vk)) |>
  ggplot(aes(x = geneID, y = V2)) +
  geom_point(pch=21) +
  geom_hline(yintercept = c(-2,0,2)*sd(Vk[,2]), col = "red"),
ncol=2)
```



We now repeat the analysis, but with the constraint $\|\mathbf{v}\|_1 < 10$. This gives a much less sparse solution with several genes with non-zero loadings.

```
spcX10 <- SPC(
  toxData[,-1] |>
```

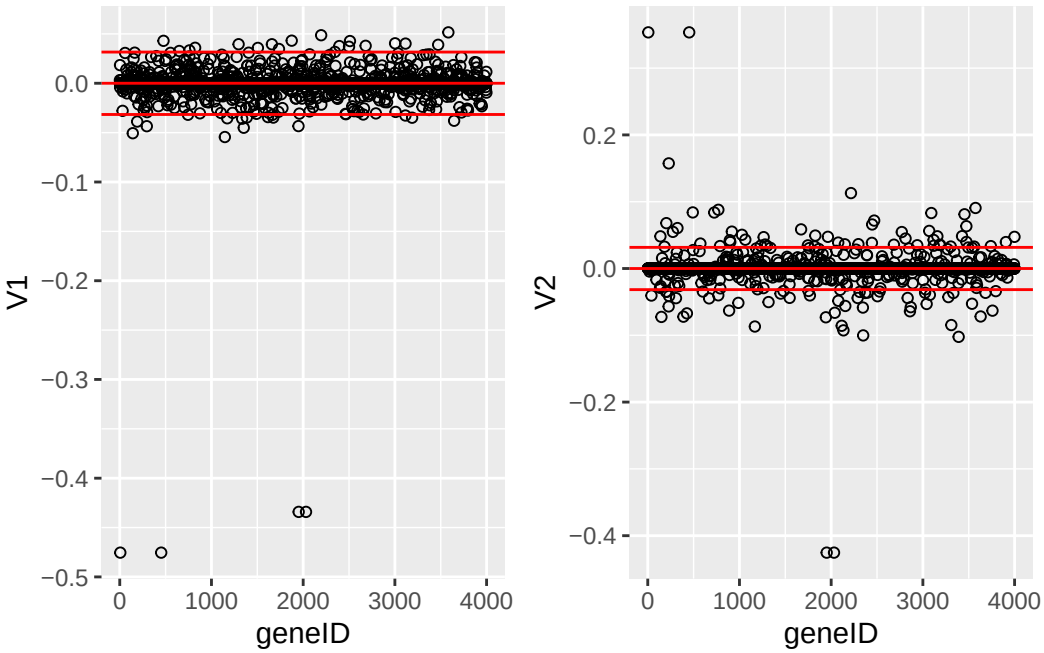
```
    as.matrix(),
  K=30,
  sumabsv = 10)
```

```
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
#> 12345678910111213141516171819 20
```

```
k <- 2
Vk <- spcX10$v[,1:k]
colnames(Vk) <- paste0("V",1:k)

grid.arrange(
  Vk |>
    as.data.frame() |>
```

```
    mutate(geneID = 1:nrow(Vk)) |>
    ggplot(aes(x = geneID, y = V1)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vk[,1]), col = "red") ,
  Vk |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vk)) |>
    ggplot(aes(x = geneID, y = V2)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vk[,2]), col = "red"),
  ncol=2)
```

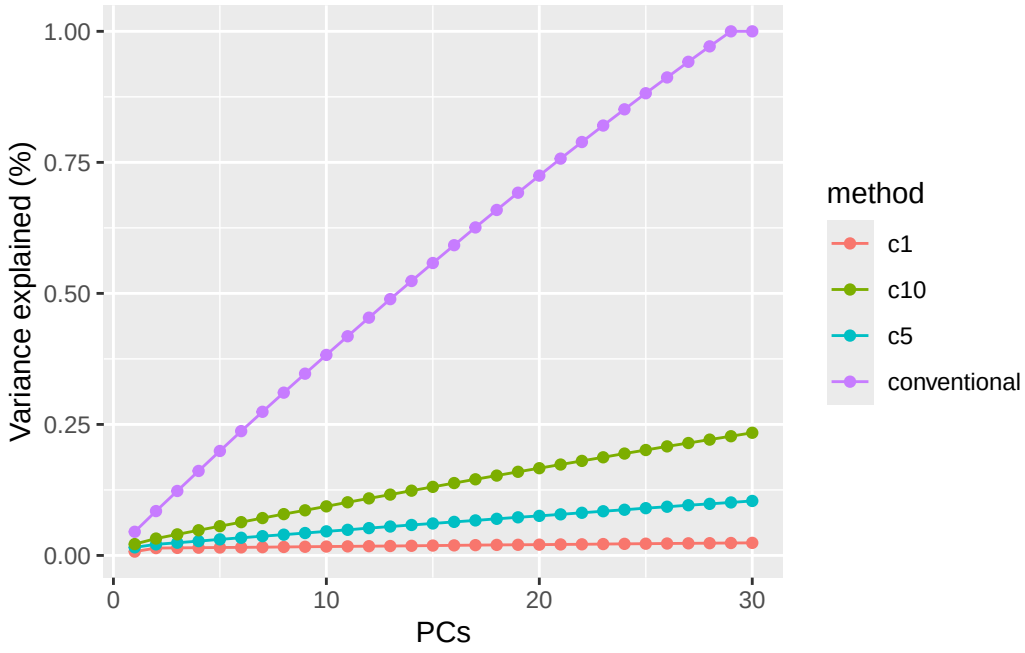


3.4 Variance explained

Variance explained for $c = 1, 5, 10$ and $+\infty$.

```
data.frame(
  PCs = 1:30,
  "c1" = spcX1$prop.var.explained,
```

```
  "c5" = spcX$prop.var.explained,
  "c10" = spcX10$prop.var.explained,
  "conventional" = cumsum(svdX$d^2)/sum(svdX$d^2)
  ) |>
  gather("method","varExpl",-1) |>
  ggplot(aes(x = PCs, y = varExpl, color = method)) +
  geom_line() +
  geom_point() +
  ylab("Variance explained (%)")
```



This graph illustrates that the percentages of variance explained by the PCs are smaller for sparser solutions. Thus in choosing an appropriate penalty parameter c_2 one should not only look at the sparseness of the solution, but also at the percentage of variance contained in the first few PCs. If this percentage is very small, then the graphs of the scores (or biplots) are no longer very informative (i.e. they will not tell the full story). Thus a compromise must be looked for.

```
library(latex2exp)

Uk5 <- spcX$u[,1:k]
Dk5 <- diag(spcX$d[1:k])
Zk5 <- Uk5%*%Dk5
```

```r
colnames(Zk5) <- paste0("Z",1:k)

spc5 <- Zk5 |>
  as.data.frame() |>
  mutate(BA = toxData |> pull(BA)) |>
  ggplot(aes(x= Z1, y = Z2, color = BA)) +
  geom_point(size = 3) +
  scale_colour_gradient2(low = "blue",mid="white",high="red") +
  geom_point(size = 3, pch = 21, color = "black") +
  ggtitle(TeX("$c_2 = 5$"))

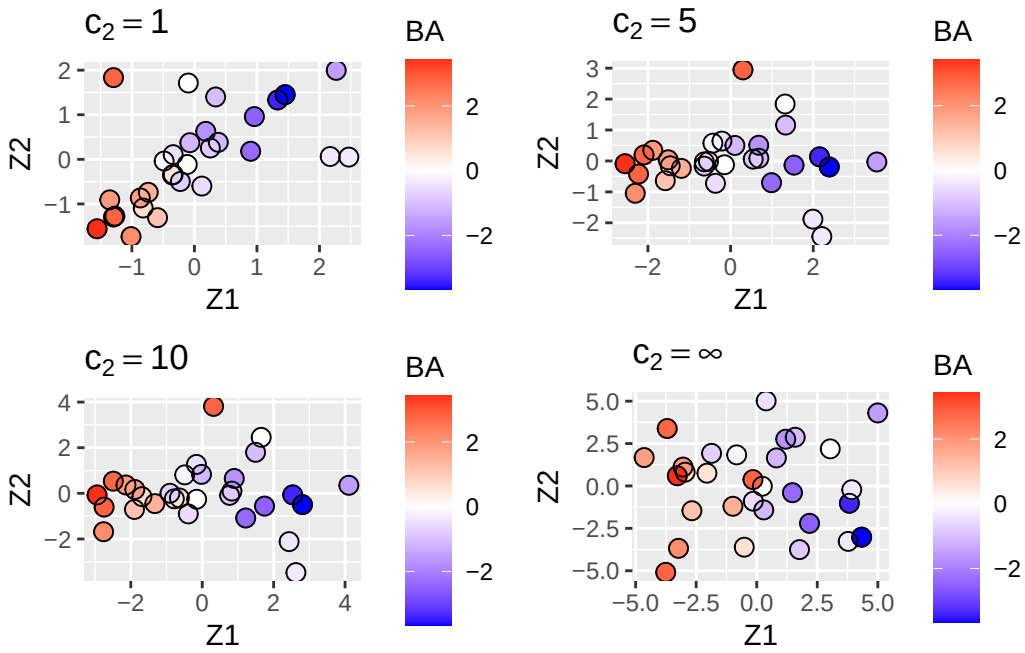
Uk1 <- spcX1$u[,1:k]
Dk1 <- diag(spcX1$d[1:k])
Zk1 <- Uk1%*%Dk1
colnames(Zk1) <- paste0("Z",1:k)

spc1 <- Zk1 |>
  as.data.frame() |>
  mutate(BA = toxData |> pull(BA)) |>
  ggplot(aes(x= Z1, y = Z2, color = BA)) +
  geom_point(size = 3) +
  scale_colour_gradient2(low = "blue",mid="white",high="red") +
  geom_point(size = 3, pch = 21, color = "black") +
  ggtitle(TeX("$c_2 = 1$"))

Uk10 <- spcX10$u[,1:k]
Dk10 <- diag(spcX10$d[1:k])
Zk10 <- Uk10%*%Dk10
colnames(Zk10) <- paste0("Z",1:k)

spc10 <- Zk10 |>
  as.data.frame() |>
  mutate(BA = toxData |> pull(BA)) |>
  ggplot(aes(x= Z1, y = Z2, color = BA)) +
  geom_point(size = 3) +
  scale_colour_gradient2(low = "blue",mid="white",high="red") +
  geom_point(size = 3, pch = 21, color = "black") +
  ggtitle(TeX("$c_2 = 10$"))

grid.arrange(spc1, spc5, spc10, pca + ggtitle(TeX("$c_2 = \\infty")), ncol = 2)
```



Finally we show the plots of the scores on the first two sparse PCs for $c_2 = 1$ (top-left), $c_2 = 5$ (top-right) and $c_2 = 10$ (bottom-left) and $c_2 = \infty$ (bottom-right, convention SVD).

Note, that the most sparse solution (top-left) shows a strong positive correlation between the first two PCs (as a consequence of the loss of orthogonality of the singular vectors).

All graphs succeed quite well in separating the toxic from the non-toxic compounds based on their gene expression.

- Hence, we may expect that the two genes identified from the most sparse solution play an important role in the development of toxic effects.
- Note, however, that our data analysis did not provide a scientific proof for this conclusion: (sparse) PCA is only an exploratory or descriptive data analysis method that helps the data-analyst to gain insight into the data.
- The fact that we found a few genes that appear to be associated with toxicity is of course a nice result.
- It is important to understand that the toxicity outcome was not used in the (sparse) PCA. We only used the toxicity to color the points in the graphs. PCA is an unsupervised method.

Acknowledgement

- Olivier Thas for sharing his materials of Analysis of High Dimensional Data 2019-2020, which I used as the starting point for this chapter.

Session info

Session info

```
#> [1] "2026-09-16 09:41:21 CEST"
```

```
#> - Session info ---------------------------------------------------------------
#>  setting  value
#>  version  R version 4.6.1 (2026-06-24)
#>  os       macOS Tahoe 26.1
#>  system   aarch64, darwin23
#>  ui       X11
#>  language (EN)
#>  collate  nl_BE.UTF-8
#>  ctype    nl_BE.UTF-8
#>  tz       Europe/Brussels
#>  date     2026-09-16
#>  pandoc   3.10.2 @ /opt/homebrew/bin/ (via rmarkdown)
#>  quarto   1.9.37 @ /usr/local/bin/quarto
#>
#> - Packages -------------------------------------------------------------------
#>  package      * version date (UTC) lib source
#>  bit            4.6.0   2025-03-06 [1] CRAN (R 4.6.0)
#>  bit64          4.8.2   2026-05-19 [1] CRAN (R 4.6.0)
#>  cli            3.6.6   2026-04-09 [1] CRAN (R 4.6.0)
#>  crayon         1.5.3   2024-06-20 [1] CRAN (R 4.6.0)
#>  curl           7.1.0   2026-04-22 [1] CRAN (R 4.6.0)
#>  digest         0.6.39  2025-11-19 [1] CRAN (R 4.6.0)
#>  dplyr        * 1.2.1   2026-04-03 [1] CRAN (R 4.6.0)
#>  evaluate       1.0.5   2025-08-27 [1] CRAN (R 4.6.0)
#>  farver         2.1.2   2024-05-13 [1] CRAN (R 4.6.0)
#>  fastmap        1.2.0   2024-05-15 [1] CRAN (R 4.6.0)
#>  forcats      * 1.0.1   2025-09-25 [1] CRAN (R 4.6.0)
#>  generics       0.1.4   2025-05-09 [1] CRAN (R 4.6.0)
#>  ggplot2      * 4.0.3   2026-04-22 [1] CRAN (R 4.6.0)
```

```
#>  glue           1.8.1   2026-04-17 [1] CRAN (R 4.6.0)
#>  gridExtra    * 2.3.1   2026-06-25 [1] CRAN (R 4.6.1)
#>  gtable         0.3.6   2024-10-25 [1] CRAN (R 4.6.0)
#>  hms            1.1.4   2025-10-17 [1] CRAN (R 4.6.0)
#>  htmltools      0.5.9   2025-12-04 [1] CRAN (R 4.6.0)
#>  jsonlite       2.0.0   2025-03-27 [1] CRAN (R 4.6.0)
#>  knitr          1.51    2025-12-20 [1] CRAN (R 4.6.0)
#>  labeling       0.4.3   2023-08-29 [1] CRAN (R 4.6.0)
#>  latex2exp    * 0.9.8   2026-01-09 [1] CRAN (R 4.6.0)
#>  lifecycle      1.0.5   2026-01-08 [1] CRAN (R 4.6.0)
#>  lubridate    * 1.9.5   2026-02-04 [1] CRAN (R 4.6.0)
#>  magrittr       2.0.5   2026-04-04 [1] CRAN (R 4.6.0)
#>  otel           0.2.0   2025-08-29 [1] CRAN (R 4.6.0)
#>  pillar         1.11.1  2025-09-17 [1] CRAN (R 4.6.0)
#>  pkgconfig      2.0.3   2019-09-22 [1] CRAN (R 4.6.0)
#>  PMA          * 1.2-4   2024-09-03 [1] CRAN (R 4.6.0)
#>  purrr        * 1.2.2   2026-04-10 [1] CRAN (R 4.6.0)
#>  R6             2.6.1   2025-02-15 [1] CRAN (R 4.6.0)
#>  RColorBrewer   1.1-3   2022-04-03 [1] CRAN (R 4.6.0)
#>  readr        * 2.2.0   2026-02-19 [1] CRAN (R 4.6.0)
#>  rlang          1.2.0   2026-04-06 [1] CRAN (R 4.6.0)
#>  rmarkdown      2.31    2026-03-26 [1] CRAN (R 4.6.0)
#>  S7             0.2.2   2026-04-22 [1] CRAN (R 4.6.0)
#>  scales         1.4.0   2025-04-24 [1] CRAN (R 4.6.0)
#>  sessioninfo    1.2.4   2026-06-04 [1] CRAN (R 4.6.0)
#>  stringi        1.8.7   2025-03-27 [1] CRAN (R 4.6.0)
#>  stringr      * 1.6.0   2025-11-04 [1] CRAN (R 4.6.0)
#>  tibble       * 3.3.1   2026-01-11 [1] CRAN (R 4.6.0)
#>  tidyr        * 1.3.2   2025-12-19 [1] CRAN (R 4.6.0)
#>  tidyselect     1.2.1   2024-03-11 [1] CRAN (R 4.6.0)
#>  tidyverse    * 2.0.0   2023-02-22 [1] CRAN (R 4.6.0)
#>  timechange     0.4.0   2026-01-29 [1] CRAN (R 4.6.0)
#>  tzdb           0.5.0   2025-03-15 [1] CRAN (R 4.6.0)
#>  vctrs          0.7.3   2026-04-11 [1] CRAN (R 4.6.0)
#>  vroom          1.7.1   2026-03-31 [1] CRAN (R 4.6.0)
#>  withr          3.0.3   2026-06-19 [1] CRAN (R 4.6.0)
#>  xfun           0.59    2026-06-19 [1] CRAN (R 4.6.0)
#>  yaml           2.3.12  2025-12-10 [1] CRAN (R 4.6.0)
#>
#>  [1] /Library/Frameworks/R.framework/Versions/4.6/Resources/library
#>  * -- Packages attached to the search path.
#>
#> ------------------------------------------------------------------------------
```